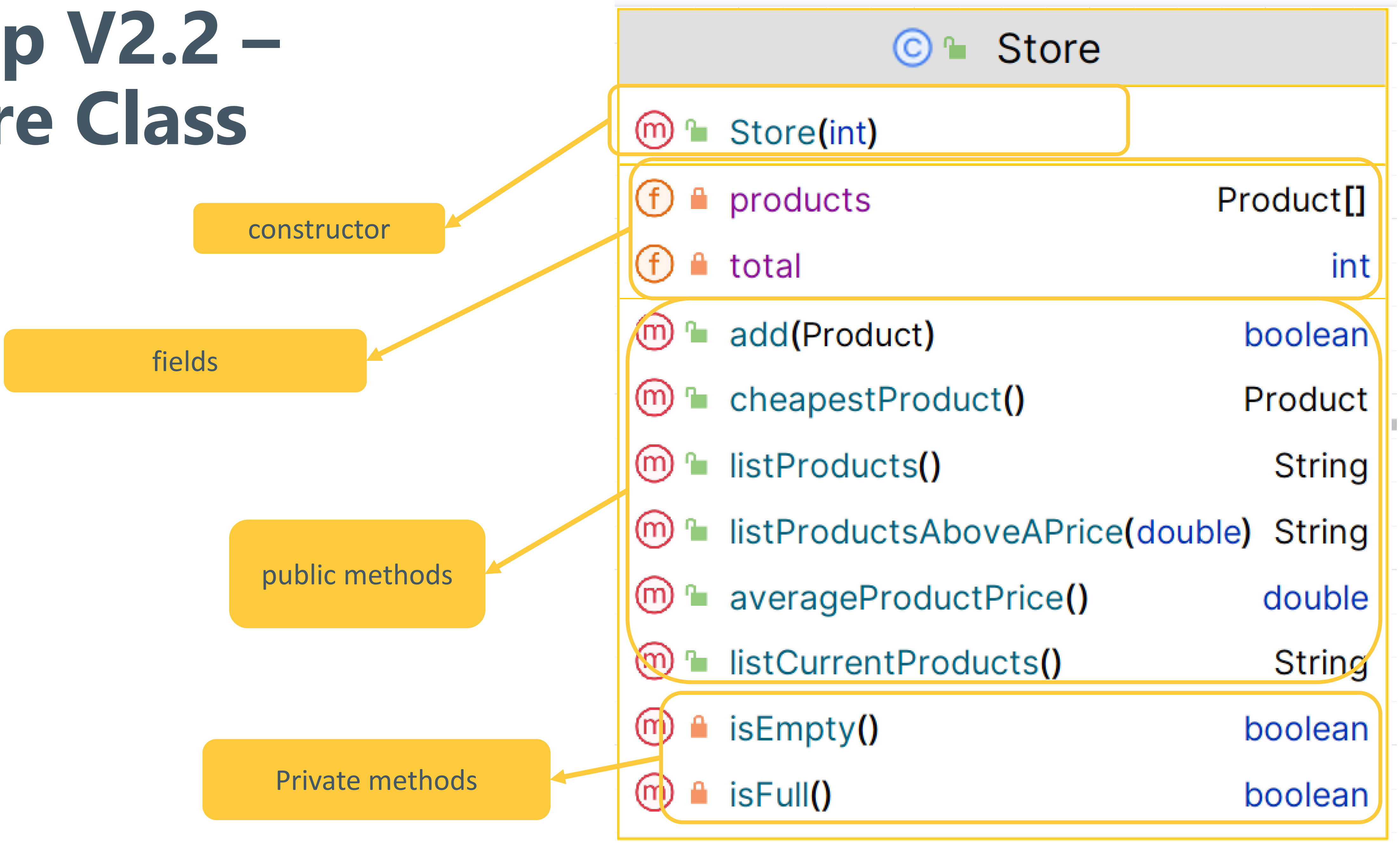
Shop v2.2 – Store

Produced by: Ms Siobhan Roche
Ms Mairead Meagher

Department of Computing and Mathematics
setu.ie

Shop V2.2 – Store Class



Shop V2.2 – Store – Fields and Constructor

```
public class Store {

    12 usages
    private Product[] products;
    10 usages
    private int total = 0;  //dual purpose.
                            // 1) number of items stored in array,
                            //  2) next available index location


    public Store(int numberItems) {

        products = new Product[numberItems];
    }
```

Shop V2.2 – Store (private methods)

```java
private boolean isFull() {

    return total == products.length;
}


private boolean isEmpty() {

    return total == 0;
}
```

getters

- **isFull()** & **isEmpty()** return state of fields.
- They are private member methods.

Shop V2.2 – Store (add() Method)

```java
public boolean add(Product product) {
    if (isFull()) {
        return false;
    } else {
        products[total] = product;
        total++;
        return true;
    }
}
```

- add() makes use of private method **isFull()**

Shop V2.2 – Store (listProducts())

```java
public String listProducts() {
    if (isEmpty()) {
        return "No products in the store";
    } else {
        String listOfProducts = "";
        for (int i = 0; i < total; i++) {
            listOfProducts += i + ": " + products[i] + "\n";
        }
        return listOfProducts;
    }
}
```

list

- toString ’type’ method listProducts() makes use of private method **isEmpty()**

Product Class

```java
public String toString()
{
    String yesOrNo;
    if (inCurrentProductLine)
        yesOrNo = "T";
    else yesOrNo = "F";
    return "Product description: " + productName
            + ", product code: " + productCode
            + ", unit cost: " + unitCost
            + ", currently in product line: " + yesOrNo;
}
```

Shop V2.2 – Store (listProductsAbovePrice)

```java
public String listProductsAboveAPrice(double price) {
    if (isEmpty()) {
        return "No Products in the store";
    } else {
        String str = "";
        for (int i = 0; i < total; i++) {
            if (products[i].getUnitCost() > price)
                str += i + ": " + products[i] + "\n";
        }
        if (str.equals("")) {
            return "No products are more expensive than: " + price;
        } else {
            return str;
        }
    }
}
```

```java
public double getUnitCost(){

    return unitCost;
}
```

Product Class

Shop V2.2 – Store (cheapestProduct)

```java
public Product cheapestProduct() {
    if (!isEmpty()) {
        Product cheapestProduct = products[0];
        for (int i = 1; i < total; i++) {
            if (products[i].getUnitCost() < cheapestProduct.getUnitCost())
                cheapestProduct = products[i];
        }
        return cheapestProduct;
    } else {
        return null;
    }
}
```

```java
public double getUnitCost(){

    return unitCost;
}
```

Product Class

Shop V2.2 – Store (listCurrentProducts)

```java
public String listCurrentProducts() {
    if (isEmpty()) {
        return "No Products in the store";
    } else {
        String listOfProducts = "";
        for (int i = 0; i < total; i++) {
            if (products[i].isInCurrentProductLine())
                listOfProducts += i + ": " + products[i] + "\n";
        }
        if (listOfProducts.equals("")){
            return "No Products are in our current product line";
        }
        else{
            return listOfProducts;
        }
    }
}
```

```java
public boolean isInCurrentProductLine() {

    return inCurrentProductLine;
}
```

Product Class

Shop V2.2 – Store (averageProductPrice)

```java
public double averageProductPrice() {
    if (!isEmpty()) {
        double totalPrice = 0;
        for (int i = 0; i < total; i++) {
            totalPrice += products[i].getUnitCost();
        }
        return totalPrice / total;
    } else {
        return -1;
    }
}
```

```java
public double getUnitCost(){

    return unitCost;
}
```

Product Class